`RecipeService` and then build and return the `RecipeSummaryViewModel` inside your endpoint handler code.

The `GetRecipes` method in `RecipeService` is conceptually simple and follows a common pattern for querying an EF Core database, as shown in figure 12.13. EF Core uses a fluent chain of LINQ commands to define the query to return on the database. The `DbSet<Recipe>` property on `AppDataContext` is an `IQueryable`, so you can use all the usual `Select()` and `Where()` clauses that you would with other `IQueryable` providers. EF Core converts these LINQ methods into a SQL statement to query the database when you call an execute function such as `ToListAsync()`, `ToArrayAsync()`, or `SingleAsync()`, or their non-async brethren.

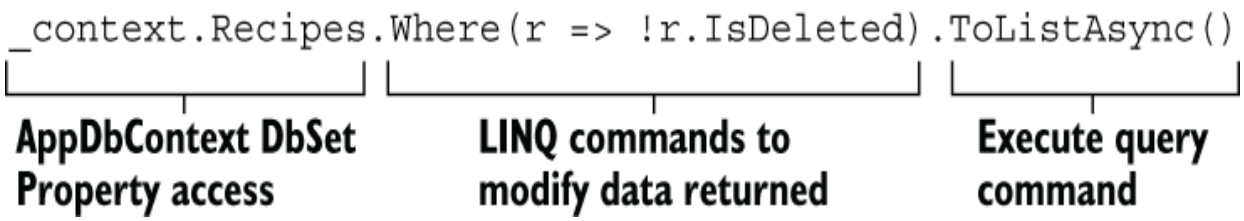

Figure 12.13 The three parts of an EF Core database query

You can also use the `Select()` extension method to map to objects other than your entities as part of the SQL query. You can use this technique to query the database efficiently by fetching only the columns you need.

Listing 12.7 shows the code to fetch a list of `RecipeSummaryViewModel`s, following the same basic pattern as figure 12.12. It uses a `Where` LINQ expression to filter out recipes marked as deleted and a `Select` clause to map to the view models. The